

Sistema Distribuido de Reconocimiento de Objetos con IA (CNN) y Consenso Raft

Examen Final – CC4P1 Programación Concurrente y Distribuida

Melissa Iman Noriega Junior Ortega
Andrew Inga

Escuela de Ciencias de la Computación – Facultad de Ciencias – UNI

2026-I

Resumen

Construimos un sistema distribuido que reconoce objetos y animales reales en tres cámaras con una **CNN escrita desde cero en NumPy** y guarda cada detección (con su foto) en un **registro replicado con Raft**. El cúmulo es heterogéneo: un nodo en **Java**, uno en **Python** y uno en **Go**, cada uno con su implementación de Raft hablando el mismo protocolo de texto sobre **sockets TCP nativos**. No usamos frameworks de mensajería ni librerías de red de terceros: solo la biblioteca base de cada lenguaje (NumPy se permite para la CNN). Un servidor de video emite los frames, el Servidor de Testeo los reconoce con la CNN (10 clases reales de CIFAR-10) e inserta al cúmulo, y los tres nodos aplican las mismas detecciones en el mismo orden. El registro se consulta desde un Cliente Vigilante de escritorio (Java) y uno móvil (Android nativo). Probamos el failover matando al líder (Python): Go es reelegido en el siguiente *term* y el registro sobrevive consistente. La CNN alcanza **40.9 %** en el test de CIFAR-10 (azar 10 %) y su entrenamiento distribuido (data-parallel con **multiprocessing**) reparte la carga entre núcleos sin perder accuracy.

1. Componentes del sistema

El sistema tiene tres piezas conectadas por el cúmulo Raft: un *Servidor de Testeo* que corre la CNN sobre las cámaras, un *cúmulo Raft* de tres nodos que mantiene el registro replicado, y un *Cliente Vigilante* que consulta ese registro. La Tabla 1 indica el lenguaje de cada componente.

Componente	Lenguaje	Función
Servidor de Testeo	Python	Corre la CNN sobre las cámaras e inserta detecciones
CNN + entrenamiento	Python	Modelo desde cero con NumPy, entrenamiento distribuido
Núcleo Raft	Java / Python / Go	Un nodo por lenguaje, mismo protocolo de texto
Máquina de estado	Go	Registro replicado de detecciones
Cliente Vigilante	Java	Interfaz que consulta el registro consistente

Cuadro 1: Componentes del sistema y lenguaje de cada uno.

2. Arquitectura

El sistema se organiza en cuatro subsistemas que se comunican por sockets TCP crudos. Un **servidor de video** emite los frames; el **Servidor de Testeo** los recibe en tres cámaras (un hilo por cámara) y los pasa por la CNN; cada detección se inserta en un **cúmulo Raft** de tres nodos que mantiene el registro replicado; y un **Cliente Vigilante** (escritorio y móvil) consulta ese registro. La Figura 1 resume el flujo.

El punto de diseño central es que el registro de detecciones no vive en una sola máquina: es una *máquina de estado replicada*. Cada detección que produce la CNN se manda al **líder** del

cúmulo como un comando; el líder lo agrega a su log y lo replica a los seguidores, y solo cuando la **mayoría** lo ha copiado se aplica al registro y se considera confirmado. Así, si un nodo cae, los otros conservan el registro completo y el sistema sigue aceptando detecciones. El líder también es el único que responde lecturas del Vigilante, para que este vea siempre un estado consistente; si el Vigilante contacta un seguidor, este lo redirige al líder.

Los tres nodos están escritos en lenguajes distintos (Java, Python y Go) pero hablan el mismo protocolo de texto (Sección 4), así que forman un solo cúmulo heterogéneo: un líder Python puede replicar en un seguidor Java o Go sin ninguna capa de traducción. Cada nodo usa concurrencia nativa para no corromper su estado: hilos con `synchronized/Lock` en Java y Python, goroutines con `Mutex` en Go.

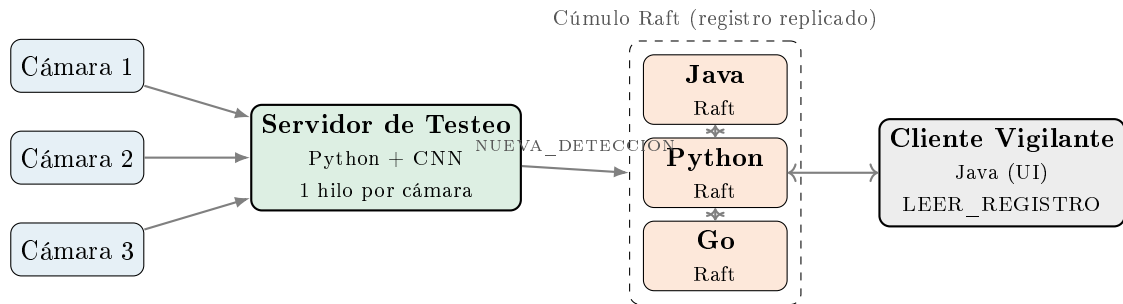


Figura 1: Arquitectura: 3 cámaras → Servidor de Testeo (Python+CNN) → cúmulo Raft heterogéneo {Java, Python, Go} → Cliente Vigilante. Toda escritura pasa por el líder y se replica por mayoría.

Concurrencia. El Servidor de Testeo levanta un hilo por cámara, de modo que las tres corren la CNN en paralelo. Cada nodo Raft protege su estado (`term` actual, voto, log, `commitIndex`) con un mutex, porque el hilo que recibe mensajes de red y el que corre el temporizador de elección tocan las mismas variables. En Go esto es una goroutine por conexión más un mutex sobre la máquina de estado; en Java y Python es un hilo por conexión con `synchronized / Lock`.

3. Consenso Raft

La implementación sigue el paper de Ongaro y Ousterhout, el mismo material del curso. Cada nodo está en uno de tres estados: **Seguidor**, **Candidato** o **Líder**.

Elección de líder. Un seguidor que no recibe latidos del líder dentro de su *timeout* (aleatorio entre 150 y 300 ms) pasa a Candidato, incrementa su `term`, vota por sí mismo y envía `REQUEST_VOTE` a los demás. El timeout aleatorio evita que los tres nodos se postulen a la vez. Un nodo concede el voto solo si el candidato está al menos tan actualizado como él (restricción de elección: se comparan `lastLogTerm` y `lastLogIndex`). Con mayoría de votos el candidato se vuelve Líder y empieza a mandar latidos.

Replicación del log. El líder recibe cada `NUEVA_DETECCION`, la agrega a su log y la propaga con `APPEND_ENTRIES`. Cada mensaje lleva `prevLogIndex` y `prevLogTerm`; el seguidor solo acepta si esos campos calzan con su propio log (*Log Matching*). Si no calzan, el líder retrocede el `nextIndex` de ese seguidor y reintenta hasta encontrar el punto común, sobrescribiendo lo que diverja.

Seguridad y commit. Una entrada se considera *committed* cuando está replicada en la mayoría. Respetamos la regla del paper: el líder solo cuenta como committed las entradas de su **term actual**; las de terms anteriores se confirman de forma indirecta al confirmarse una posterior. Recién cuando

una entrada está committed se aplica a la máquina de estado (el registro). Así los tres nodos aplican las mismas detecciones en el mismo orden.

El siguiente extracto muestra cómo un nodo decide conceder o no su voto al recibir `REQUEST_VOTE`, con la restricción de elección que compara quién tiene el log más actualizado.

```

1 al recibir REQUEST_VOTE(term, candId, lastLogIdx, lastLogTerm):
2     con mutex bloqueado:
3         si term < termActual: # candidato atrasado
4             responder VOTE(termActual, granted=0)
5         si term > termActual: # hay un term mas nuevo
6             termActual = term; votadoPor = null; volverASeguidor()
7         # restriccion de eleccion: el candidato debe estar al dia
8         alDia = (lastLogTerm > miUltimoTerm) o
9             (lastLogTerm == miUltimoTerm y lastLogIdx >= miUltimoIdx)
10        si (votadoPor == null o votadoPor == candId) y alDia:
11            votadoPor = candId; reiniciarTimeout()
12            responder VOTE(termActual, granted=1)
13        sino:
14            responder VOTE(termActual, granted=0)

```

Listing 1: Lógica de voto al recibir `REQUEST_VOTE` (misma en los 3 nodos).

4. Protocolo de texto

Todos los mensajes son una línea de texto terminada en salto de línea, con campos separados por `|`. Los tres lenguajes hablan exactamente el mismo formato, lo que hace posible un cúmulo heterogéneo sobre TCP crudo. Hay dos grupos: los mensajes de Raft y los del registro/cliente.

```

1 # --- Raft ---
2 REQUEST_VOTE|term|candidateId|lastLogIndex|lastLogTerm
3 VOTE|term|granted
4 APPEND|term|leaderId|prevLogIndex|prevLogTerm|commitIndex|entries
5 APPEND_OK|term|success|matchIndex
6
7 # --- Registro y cliente ---
8 NUEVA_DETECCION|tipo,imagenRef,fechaHora,camara
9 LEER_REGISTRO
10 REGISTRO|det1;det2;...
11 REDIRECT|host:puerto
12 OK

```

Listing 2: Protocolo de texto compartido (Raft + registro).

Un detalle: si un cliente o el Servidor de Testeo le escribe a un seguidor, este responde `REDIRECT` con `host:puerto` apuntando al líder actual, en vez de rechazar la escritura. Así el Servidor de Testeo siempre termina escribiendo en el líder aunque cambie tras una elección. El campo `entries` de `APPEND` lleva cero o más detecciones serializadas; cuando va vacío, el mensaje funciona como latido (*heartbeat*) que mantiene vivo al líder y reinicia el temporizador de elección de cada seguidor.

5. Modelo de IA: CNN desde cero

La CNN clasifica objetos y animales reales de 32×32 (RGB) del dataset CIFAR-10: avión, auto, pájaro, gato, ciervo, perro, rana, caballo, barco y camión (10 clases). Está escrita a mano en NumPy, sin librerías de deep learning. La arquitectura es:

Conv $8@3 \times 3$ + ReLU + MaxPool $\rightarrow$ Conv $16@3 \times 3$ + ReLU + MaxPool $\rightarrow$ Flatten $\rightarrow$ Densa $\rightarrow$ Softmax.

El *forward* y el *backward* (backpropagation) están implementados a mano. La convolución usa `im2col`: se reordena la imagen en columnas para que la convolución quede como una sola multiplicación de matrices, que es mucho más rápida que iterar píxel por píxel. Se normaliza por canal

(media/std), sin lo cual el gradiente se estanca y la red no aprende. Los pesos entrenados se guardan en un `.npz` y el Servidor de Testeo los carga al arrancar (sin internet). En el test oficial de CIFAR-10 (10 000 imágenes) la red llega a **40.9 % de accuracy**; el azar con 10 clases es 10 %, así que reconoce objetos reales cerca de 4× sobre el azar. Una CNN chica hecha a mano en NumPy no alcanza el 90 % en CIFAR: 40.9 % es el resultado honesto y esperado, con objetos reales.

Entrenamiento distribuido. Entrenamos en paralelo con `multiprocessing` en esquema *data-parallel*: se parte el lote entre los *workers*, cada uno calcula el gradiente sobre su parte y se promedian los gradientes (all-reduce) antes de actualizar los pesos. Con esto la carga de cómputo se reparte entre núcleos y el entrenamiento es más rápido sin perder accuracy respecto al secuencial (el promediado de gradientes da el mismo modelo).

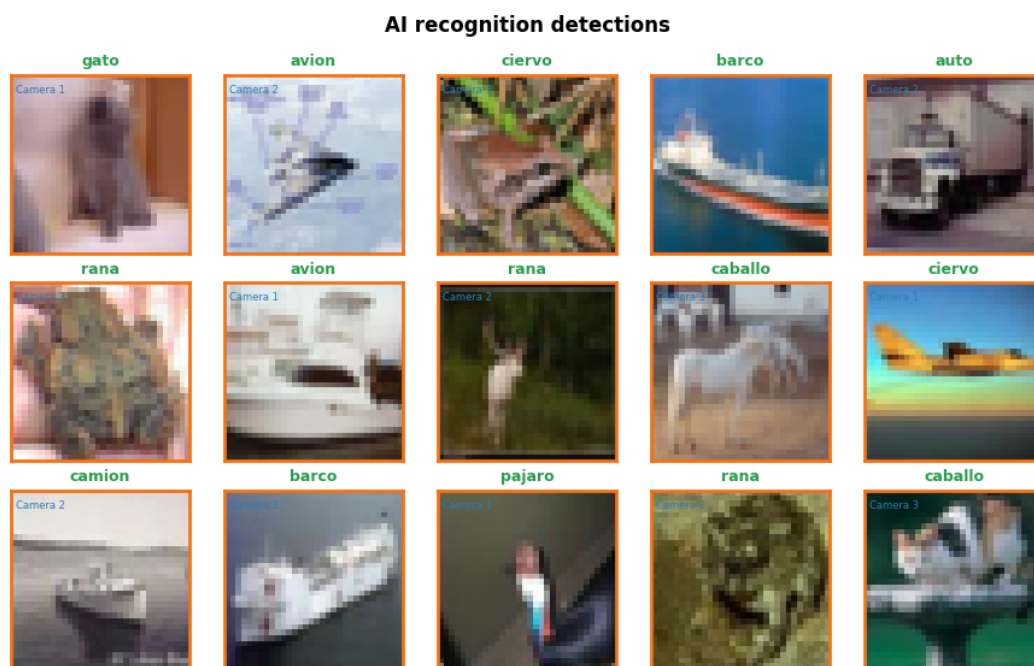


Figura 2: Objetos y animales reales reconocidos por la CNN (etiqueta en verde, cámara en azul).

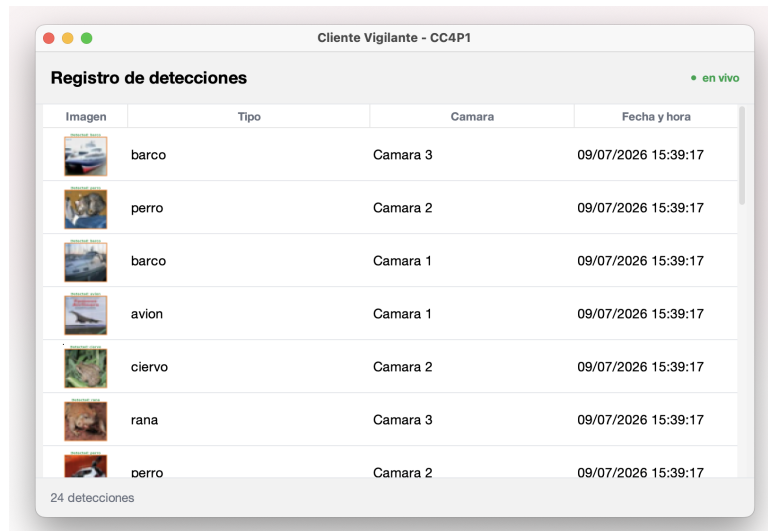
6. Pruebas end-to-end y tolerancia a fallos

Levantamos el cúmulo con los tres nodos (Java + Python + Go) y el Servidor de Testeo con las 3 cámaras. El sistema funciona de punta a punta:

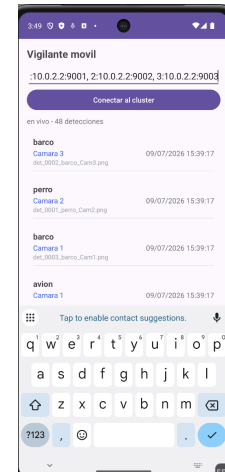
- El cúmulo heterogéneo **elige líder** sin intervención.
- La CNN **reconoce en las 3 cámaras** en paralelo (un hilo por cámara), con **88–100 % de acierto** por cámara.
- Los tres nodos aplican las **mismas 75 detecciones en el mismo orden**: el registro queda idéntico en Java, Python y Go.

Consistencia del registro. La forma de verificar que la réplica quedó bien fue comparar el registro de los tres nodos al final de la corrida: pedimos `LEER_REGISTRO` a cada uno y comparamos las 75 detecciones. Las tres respuestas `REGISTRO|...` salieron idénticas, con las detecciones en el mismo orden. Esto confirma que Raft no solo replicó el contenido, sino también el orden, que es lo que garantiza la máquina de estado determinista: aplicar el mismo log en el mismo orden lleva a los tres nodos al mismo estado.

Failover. Para probar tolerancia a fallos matamos al líder mientras el sistema seguía insertando. En esa corrida el líder era el nodo Python. Al caer, los seguidores dejan de recibir latidos, salta el timeout de elección y **Go es reelegido líder en el siguiente *term***. El registro sobrevive consistente (ninguna detección committed se pierde) y se sigue insertando con normalidad. Esto ejercita justo la parte difícil de Raft: cambiar de líder sin corromper ni reordenar el log ya confirmado.



(a) Cliente Vigilante de escritorio (Java + FlatLaf)



(b) Cliente Vigilante móvil (Android nativo)

Figura 3: El registro replicado, con la foto de cada detección, visto desde el Cliente Vigilante de escritorio y el móvil (ambos leen el mismo cúmulo por sockets).

7. Cómo ejecutar

Todo es local, sin internet. Se entrena la CNN una vez para generar los pesos **.npz**; luego se levantan los tres nodos Raft (cada uno en su lenguaje) y por último el Servidor de Testeo y el Cliente Vigilante.

1. Entrenar la CNN (Python + NumPy) → guarda pesos en **.npz**.
2. Arrancar los 3 nodos Raft: Java, Python y Go, cada uno con su puerto. Escuchan en 0.0.0.0, así que sirven tanto en localhost como en LAN/WIFI.
3. Arrancar el Servidor de Testeo (Python), que carga los pesos y abre un hilo por cámara.
4. Arrancar el Cliente Vigilante (Java) para ver el registro replicado.

8. Conclusiones y trabajo futuro

- Armamos un cúmulo **poliglota** (Java + Python + Go) donde los tres nodos corren su propia implementación de **Raft a mano** sobre TCP crudo, cumpliendo la restricción de no usar frameworks ni MQ.
- La **CNN desde cero en NumPy** reconoce 10 objetos y animales reales de CIFAR-10 con 40.9 % en test (azar 10 %), y su **entrenamiento distribuido** reparte la carga entre núcleos sin perder accuracy.
- El sistema es consistente de punta a punta: las detecciones se aplican en el mismo orden por los tres nodos, y **failover** verificado (muere el líder Python, Go es reelegido, el registro sobrevive).